VDK/TLM Creation

Rapid Training

X-2025.06

Module 2: Lab D

Test and Coverage for the SPI Controller Model

© 2025 Synopsys, Inc.

Table of Contents

Lab Overview 2

Task-1: Setup and Open the SPI project 2

Task-2: Unit Testing 3

Task-3: Line Coverage 9

Task-4: Functional Coverage 12

Lab Overview

In the previous labs, Module2LabB and Module2LabC, we had implemented the base and incremental versioned functionality of the Serial Protocol Interface (SPI) controller peripheral – the task for this lab is to test the model using the Unit Testing framework of TLM Creator, including measuring the functional coverage.

Please refer to the SPI Controller Specification for interface and behavior of this training example model.

Task-1: Setup and Open the SPI project

***Motivation**: In order to complete this training successfully there are few setup instructions that are required. This has to do with dependencies and configuration options required to complete the exercise.*

1. If you have completed the previous lab (Module 2 Lab A), you can continue with your existing TLM Creator project and skip reading the rest of this task and continue with *`Task-2`*.

   Otherwise, if want to start from a prepared starting point for this lab, use the following steps to create the starting point.

   i. Open a console and invoke Virtualizer studio from the Synopsys Virtual Prototyping installation.

   ```
   % source <installDir>/SLS/windows/setup.bat vi[rtualizer] (Windows)

   % source <installDir>/SLS/linux/setup.csh|sh vi[rtualizer] (Linux)
   ```

   This sourcing option picks up the Virtualizer *Elite* or *Base* license in that order, whichever is available. For more details on other options, refer to the VP documentation.

   i. Go to the `<Lab-data>/data/module_2` directory and start Virtualizer Studio.

   ```
   vs -d start_Module2LabD &
   ```

2. Close the welcome screen and import the starting point TLM Creator Project by clicking ***File*** → ***Import…*** menu option.

   i. In the ***Import*** dialog, select ***General*** → ***Existing Projects into Workspace*** to select what you want to import.

   ii. Click ***Next*** to get to the ***Import Projects*** dialog. Browse to the `timer` directory in the `start_Module2LabD` directory and click ***Finish*** (Make sure you are in the right directory – it's easy to end up in the wrong directory!).

   iii. Select the `SPICtrl.tlmc` file in the Project Explorer and click ***Generate Sources Code*** from the ***Build*** toolbar button options. Also, generate the documentation using the ***Generate Documentation*** option.

This will provide you with the starting point SPI Controller model for this lab.

Task-2: Unit Testing

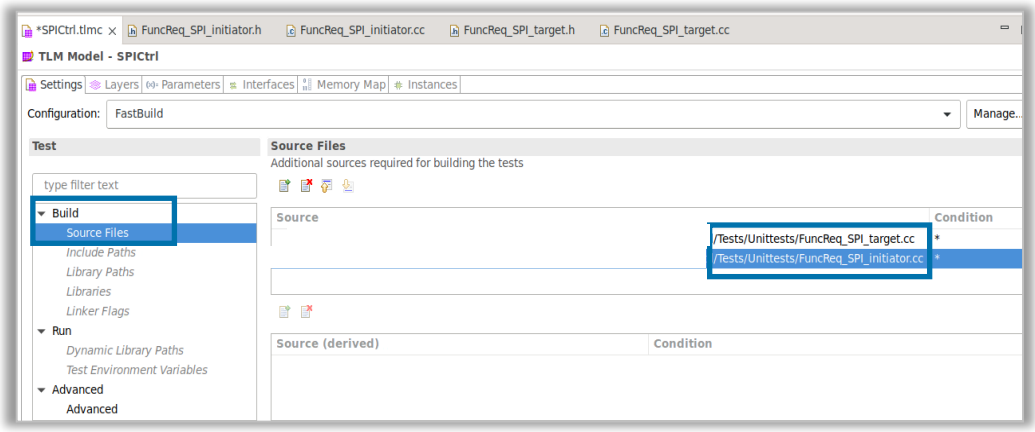
***Motivation**: This task aims at testing the SPI Controller using the TLM Unit Testing solution. It is not the objective to perform an exhaustive testing of the component but to illustrate the "proxy" based approach used for the creation of the tests. Moreover, we will split the tests in two functional requirements, each with own header and implementation files. We will use the new capability to add source files just for the test project.*

3. First, create 4 new files in the `Tests/Unittests` folder to implement the following two functional requirements:

   i. Functional requirement to test the SPI controller configured as initiator: `FuncReq_SPI_initiator.h` and `FuncReq_SPI_initiator.cc`

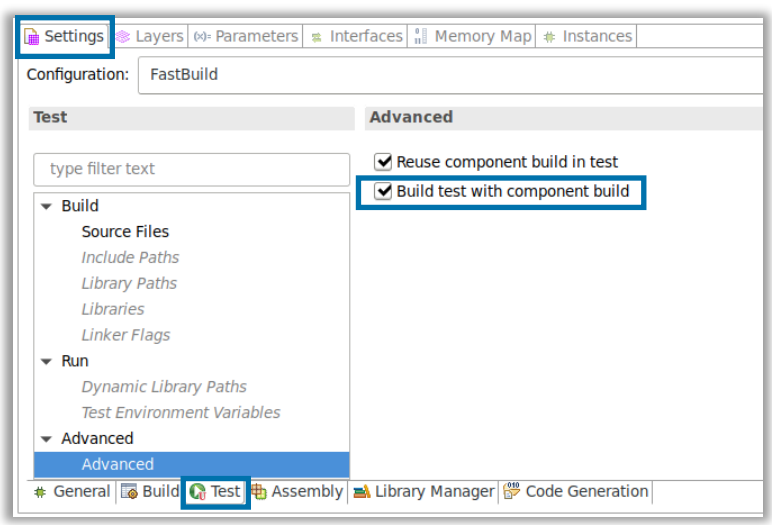
   ii. Functional requirement to test the SPI controller configured as target: `FuncReq_SPI_target.h` and `FuncReq_SPI_target.cc`

4. Click to open the ***Test*** page of the ***Settings*** tab. Click to open the ***Source Files*** settings under the ***Build*** option. Add new entries for the implementation files (`.cc`) in the ***Source*** field as follows.



We are now ready for testing the component.

Note: We disabled the automatic building of the test components to save some build/compilation time in an earlier lab module. To re-enable this option, click the ***Build test with component build*** checkbox in the ***Advanced*** settings in the ***Test*** page of the ***Settings*** tab.



5. Open and modify the `SPICtrlTest.cc` file as follows (delete all previous content):

```
#include "FuncReq_SPI_initiator.h"
#include "FuncReq_SPI_target.h"
using namespace scml2::testing;

SCML2_REGISTER_TEST_GROUP_WITH_PARAMETERS (FuncReq_SPI_initiator,
   parameter_configurations(parameter_configuration("Initiator")("version",
   10003)("enable_check_clocks", false)("target_id", 2)("fifo_size", 5)));
SCML2_REGISTER_TEST_GROUP_WITH_PARAMETERS( FuncReq_SPI_target,
   parameter_configurations(parameter_configuration("Target")("version",
   10003)("enable_check_clocks", false)("target_id", 2)("fifo_size", 5)));
SCML2_TESTING_PORT_TAG(FuncReq_SPI_initiator, bus_reg_in, "TLM2_GP");
SCML2_TESTING_PORT_TAG(FuncReq_SPI_target, bus_reg_in, "TLM2_GP");
```

6. Modify the `FuncReq_SPI_initiator.h` file as follows:

```
#include <SPICtrlTestHarness.h>
using namespace scml2::testing;

class FuncReq_SPI_initiator: public SPICtrlTestHarness {
  // Register the test class and all test methods
  SCML2_BEGIN_TESTS(FuncReq_SPI_initiator);
  SCML2_TEST(testInitiateInitiatorTransfer);
  SCML2_END_TESTS();
  FuncReq_SPI_initiator();
  // initialize is...
  virtual void initialize();
  // Called before running each testcase
  // Reset your model here.
  virtual void setup();
  // Called after running each testcase
  virtual void teardown() {
  }
  // Called at the end of the complete testrun
  virtual void shutdown() {
  }
  void testInitiateInitiatorTransfer();
};
```

The `FuncReq_SPI_target.h` is same as the master header file with `FuncReq_SPI_target` as class name and declaring the test `testInitiateTargetTransfer` instead. Copy the following code in `FuncReq_SPI_target.h` file.

```
#include <SPICtrlTestHarness.h>
using namespace scml2::testing;

class FuncReq_SPI_target : public SPICtrlTestHarness {
  // Register the test class and all test methods
  SCML2_BEGIN_TESTS(FuncReq_SPI_target);
  SCML2_TEST(testInitiateTargetTransfer);
  SCML2_END_TESTS();
  FuncReq_SPI_target();
  // initialize is...
  virtual void initialize();
  // Called before running each testcase
  // Reset your model here.
  virtual void setup();
  // Called after running each testcase
  virtual void teardown() {
  }
  // Called at the end of the complete testrun
  virtual void shutdown() {
  }
  void testInitiateTargetTransfer();
};
```

7. The `FuncReq_SPI_initiator.cc` is implemented as below. Edit the `initialize` and `setup` functions with the clock, reset and SPI configuration. Note that the SPI proxy is registered as slave with ID 2.

```
#include "FuncReq_SPI_initiator.h"

FuncReq_SPI_initiator::FuncReq_SPI_initiator() {
    // Set parameter values here
}
void FuncReq_SPI_initiator::initialize() {
      enable_logging();
      this->rst_in.set_active_level(false);
      this->rst_in.set_duration(sc_core::sc_time(100, SC_NS));
      this->clk_in.set_period_ps(10000); // 10 NS (100 MHz)
      this->spi_clk.set_period_ps(1000000); // 1 US (1 MHz)
}
void FuncReq_SPI_initiator::setup() {
      this->Serial_IO.clear();
      this->rst_in.do_reset_pulse();
      this->Serial_IO.register_as_slave(2);
      sc_core::sc_time bit_time = sc_core::sc_time(1000000, SC_PS);
      this->Serial_IO.set_bit_time( bit_time );
}
```

Implement the `testInitiateInitiatorTransfer` test. Note how in the first part of the test the SPI controller is configured and data is pushed on the *TXFIFO* register. That should be enough to initiate a SPI transfer. In the second part of the test, we wait for the SPI transfer to happen and check its values. We rely on the SPI proxy capabilities for this.

```
void FuncReq_SPI_initiator::testInitiateInitiatorTransfer() {
  // Configure the SPICtrl registers and initiate a transfer
  this->bus_reg_in.write32(MCR, 0x00000041); // Configured as Master and strobe select
  this->bus_reg_in.write32(CTAR+0, 0x0000010F); // Frame size 16 bits, Preescaler 1
  this->bus_reg_in.write32(PUSHR, 0xAAAA0200); // CTAR 0, PCS 2, TXDATA 0xAAAA
  // Wait for the transfer to start
  if( this->Serial_IO.start_received_transfer_happened() == false) {
      sc_core::wait(this->Serial_IO.get_start_received_transfer_cb_event());
  }
  SCML2_ASSERT_THAT(true, "SPI transfer received");
  // get and check the received data
  unsigned int data = this->Serial_IO.get_data();
  SCML2_ASSERT_THAT((data == 0xAAAA), "SPI data received is correct");
  // Initiate the SPI response
  this->Serial_IO.start_slave_transfer(0xBBBB, this->Serial_IO.get_length());
  // Wait for the transfer to end
  if( this->Serial_IO.end_received_transfer_happened() == false) {
      sc_core::wait(this->Serial_IO.get_end_received_transfer_cb_event());
  }
  SCML2_ASSERT_THAT(true, "SPI transfer completed");

  // check that the response is received correctly
  bool status;
  data =  this->bus_reg_in.read32(POPR, &status);
  SCML2_ASSERT_THAT((data == 0xBBBB), "Response data is correct");
}
```

8. Next, the `FuncReq_SPI_target.cc` is implemented as below. Edit the `initialize` and `setup` functions with the clock, reset and SPI configuration. Notice that the SPI proxy is registered as master now:

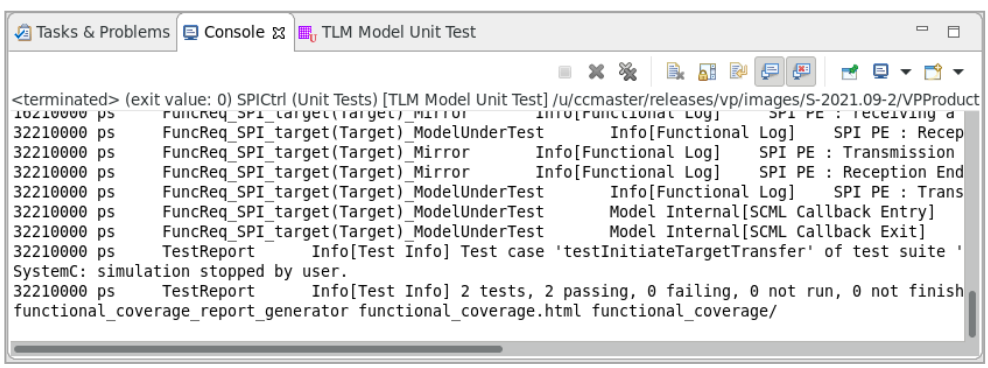
```
#include "FuncReq_SPI_target.h"

FuncReq_SPI_target::FuncReq_SPI_target() {
      this->target_id = 2;
}
void FuncReq_SPI_target::initialize() {
      enable_logging();
      this->rst_in.set_active_level(false);
      this->rst_in.set_duration(sc_core::sc_time(100, sc_core::SC_NS));
      this->clk_in.set_period_ps(10000); // 10 NS (100 MHz)
      this->spi_clk.set_period_ps(1000000); // 1 US (1 MHz)
}
void FuncReq_SPI_target::setup() {
      this->Serial_IO.clear();
      this->rst_in.do_reset_pulse();
      this->Serial_IO.register_as_master();
}
```

Implement the `testInitiateTargetTransfer` test. Notice how first the SPI controller is configured and response data is pushed on the *TXFIFO* register. Then, we explicitly initiate a SPI transfer. Then we wait for the SPI response to happen and check its values. We rely on the SPI proxy capabilities for this.

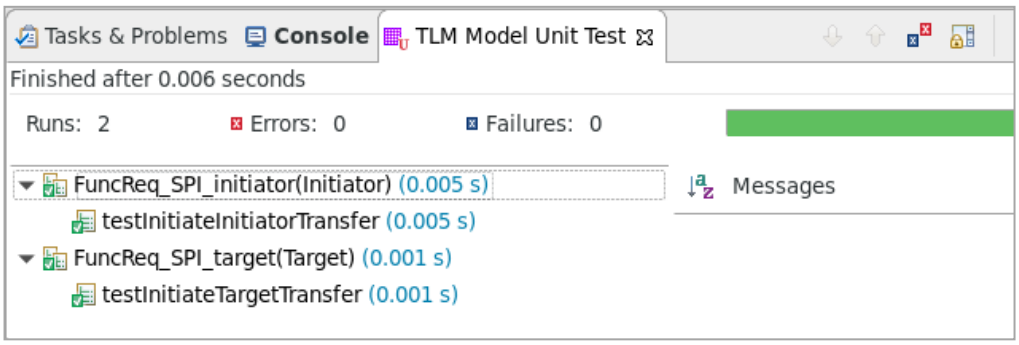
```
void FuncReq_SPI_target::testInitiateTargetTransfer() {
    // Configure the SPICtrl registers and store the response value
    this->bus_reg_in.write32(MCR, 0x00000040); // Configure as Slave and Strobe select
    this->bus_reg_in.write32(PUSHR, 0xCCCC); // Response data 0xCCCC
    // Initiate the SPI transfer (as master)
    this->Serial_IO.set_slave_select_type_strobe_select();
    sc_core::sc_time bit_time = sc_core::sc_time(1000000, SC_PS); // 1 US
    this->Serial_IO.set_bit_time( bit_time );
    this->Serial_IO.start_master_transfer(0xDDDD, 16, 2); // slave = 2
    // Wait for the response to start
    if( this->Serial_IO.start_received_transfer_happened() == false) {
       sc_core::wait(this->Serial_IO.get_start_received_transfer_cb_event());
    }
    // Check that the response data is correct
    unsigned int data = this->Serial_IO.get_data();
    SCML2_ASSERT_THAT((data == 0xCCCC), "SPI data received is correct");
    // Wait for the response to end (after delay = bit_time*16)
    if( this->Serial_IO.end_received_transfer_happened() == false) {
       sc_core::wait(this->Serial_IO.get_end_received_transfer_cb_event());
    }
    SCML2_ASSERT_THAT(true, "SPI transfer completed");
    // check time elapsed
    // check that the send data is received correctly
    bool status;
    data = this->bus_reg_in.read32(POPR, &status);
    SCML2_ASSERT_THAT((data == 0xDDDD), "SPI data is correct");
}
```

9. Save and rebuild the component (including the unit testing framework). Run the tests using the ***Run As > TLM Model Unit Test (Test Run)*** option.

In the ***Console*** tab we can observe the details of the tests execution. A total of 2 passing tests and 7 checks (assertions) has been carried out.



Similarly, the ***TLM Model Unit Test*** tab shows a more graphical view of the test run. Note that the tests are grouped per test group Initiator or Target.



Task-3: Line Coverage

Motivation*: Next to the functional coverage, TLM Creator supports also more traditional line and function coverage. The TLM Creator line and function coverage is based on lcov. To enable it, the lcov executable needs to be available in the path.* ***This feature is supported only on Linux.***

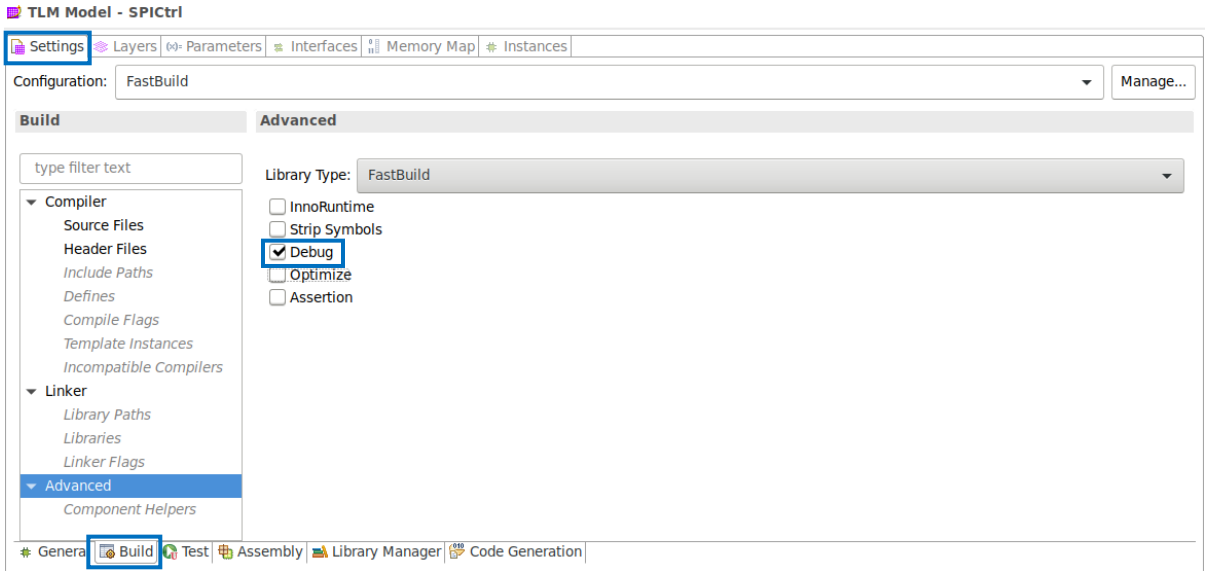
The line and function coverage are not enabled automatically. To enable it, use the following steps:

i. Activate ***Debug*** configuration.

ii. Configure ***Run*** configuration.

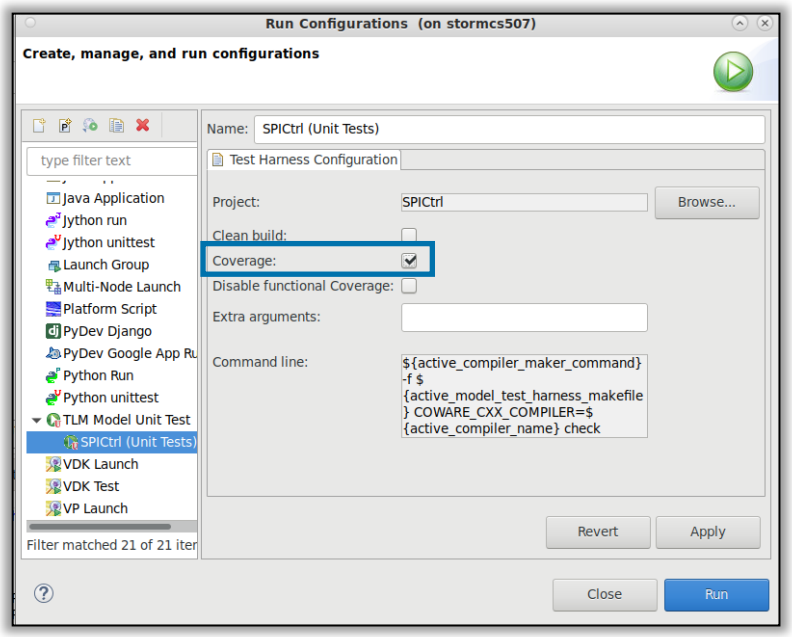
iii. Execute unit tests.

10. Open the ***Build*** page of the ***Settings*** tab. Open the ***Advanced*** settings and click to enable `Debug` configuration and disable `Optimize` setting.

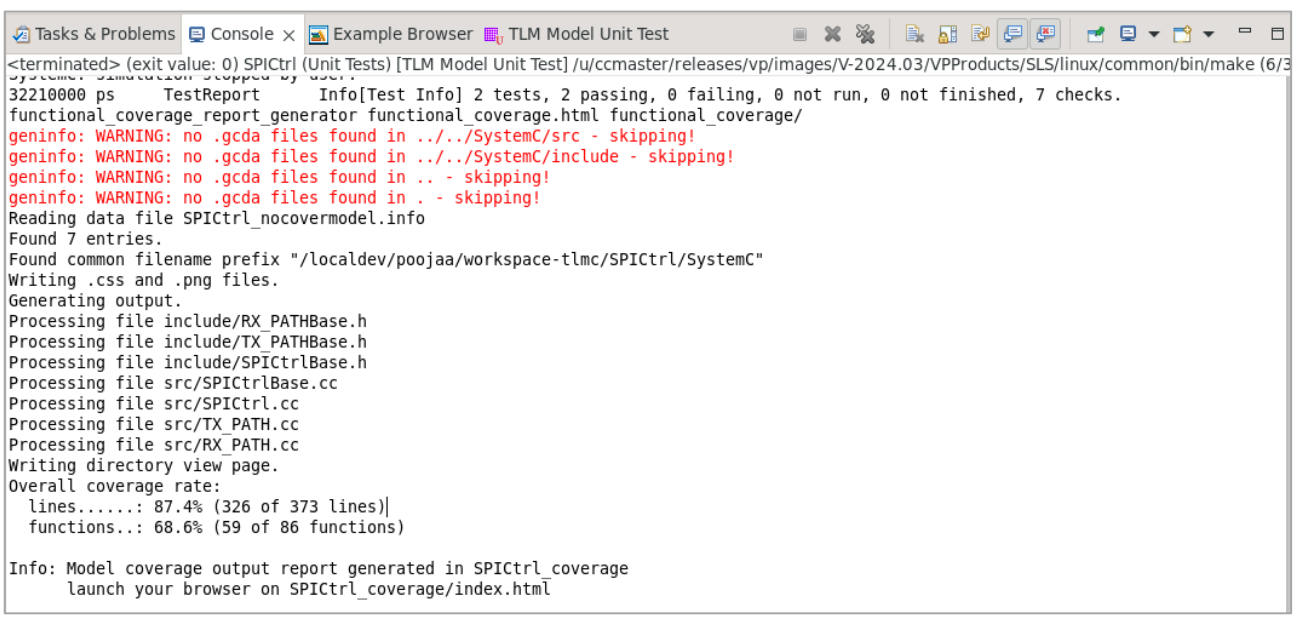


11. Save and Build the project with the default settings.

12. Next, go to the ***Run*** menu and select ***Run Configurations…*** option. In the ***Run Configuration dialog***, select the ***SPICtrl (Unit Tests)*** entry under the ***TLM Model Unit Test*** entry on the left side and check the ***Coverage*** option on the right pane of the window. The command line will be extended by `ENABLE_COVERAGE=1`.

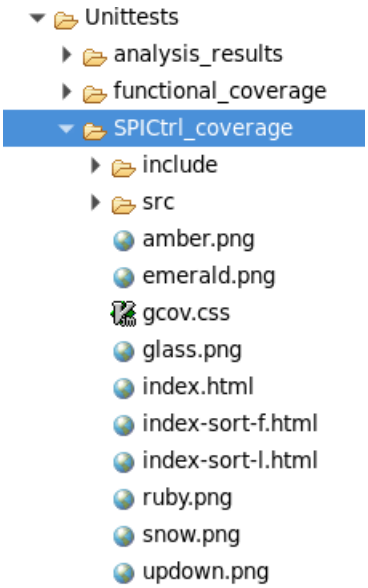


To execute again the unit tests, click ***Apply***, and then ***Run***. Ignore the generated warnings.



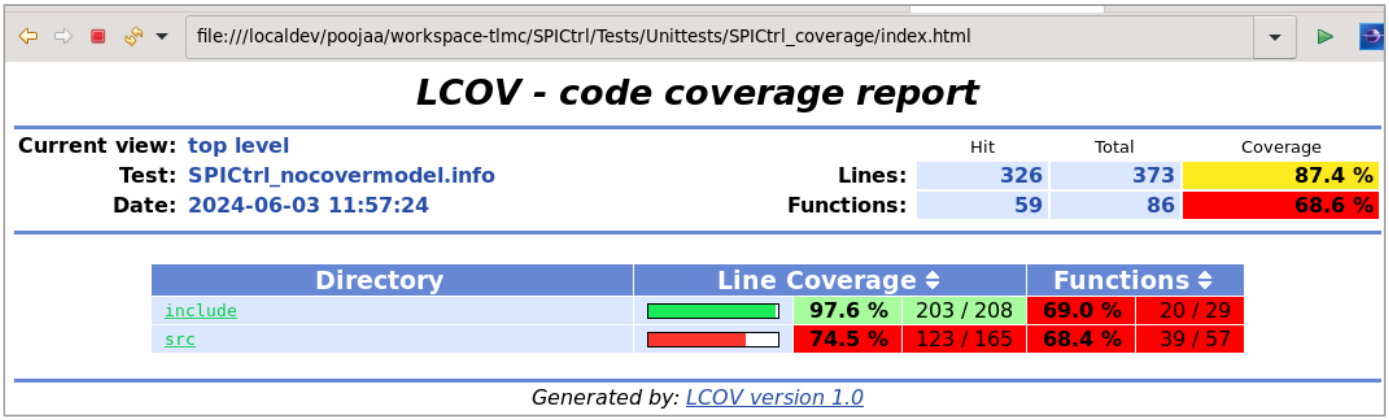
The line coverage report gets generated in the `SPICtrl/Tests/Unittests/SPICtrl_coverage` folder under 2 separate folders, `include` and `src`.

Note: Depending on the model size the lcov tool, extracting the line coverage data, may take shorter or longer and generated reports show up only after that. Wait until the status button turns from red to grey (that is till Simulation/Extraction done) to view the reports.

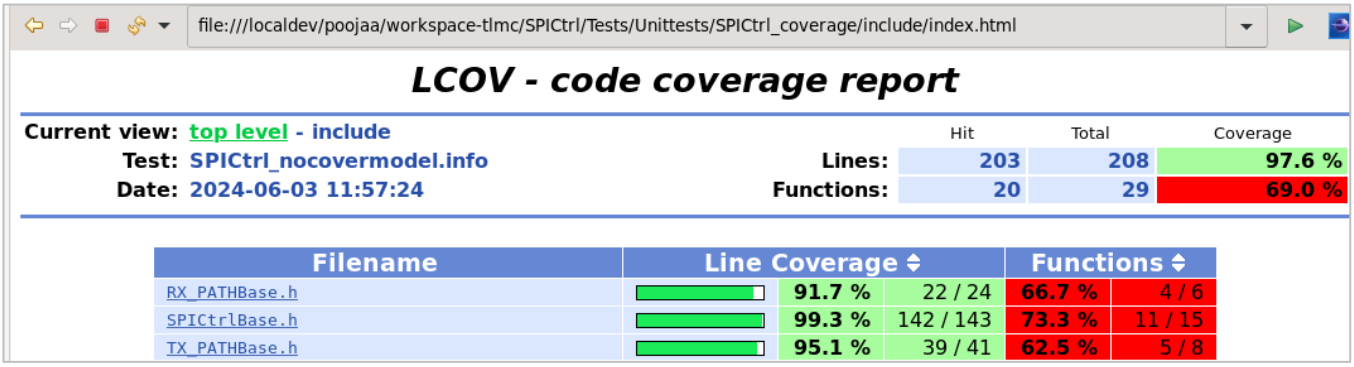


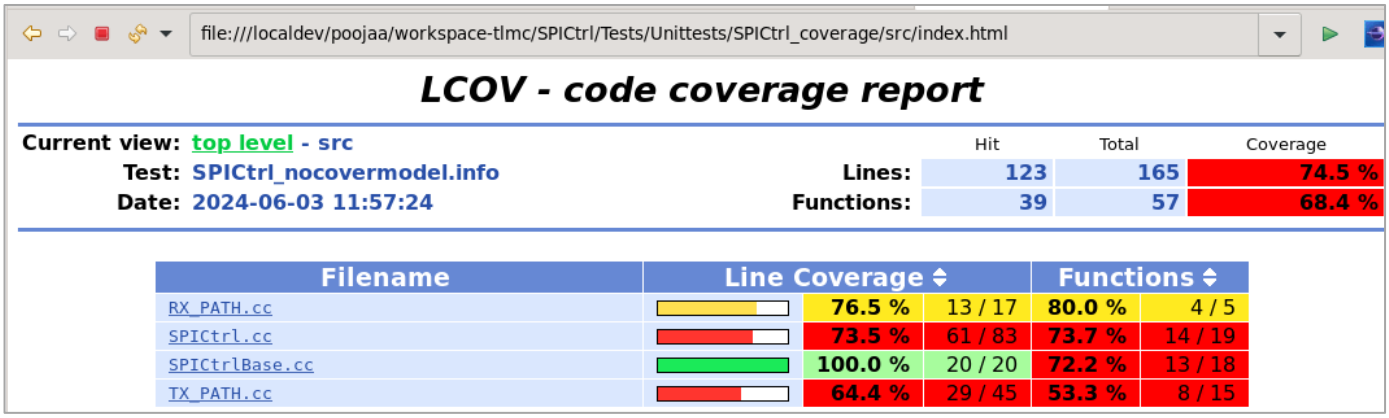
13. To view generated report, locate the `SPICtrl_coverage/index.html` file and double-click to open. The HTML file opens in the VS browser itself for the ease of reviewing.

The LCOV – code coverage report gives you an overview of a number of hit lines and functions. You can browse through the directories and files and see the coverage details.



14. Click to open the `SPICtrl_coverage/src/index.html` file. Click on any of the `*.cc` files to open them - you can check every line whether it was executed or not.





The lines with a leading 0 were not executed, i.e. covered. Below you see that the highlighted lines of the file `TX_PATH.cc` are not covered. It is up to you to decide whether the unit test needs to be extended to cover it or whether it is some dead code that can be removed, or whether you just ignore it for now.

```
          :
39        : }
40      0 : void TX_PATHType::handle_TXFIFO_almost_full() {
41      0 :   TX_FULL = true;
42      0 :   }
43      4 : void TX_PATHType::handle_TXFIFO_new_item_available() {
44      4 :         if (m_isDriver) {
45      2 :                 newItemEvent.notify(clk_period);
46        :         }
47      4 : }
48      0 : void TX_PATHType::handle_TXFIFO_overrun(const unsigned int& value) {
49        :   // FIXME: Default implementation. Implement this.
50      0 : }
51      4 : void TX_PATHType::handle_TXFIFO_almost_empty() {
52      4 :   TX_EMPTY = true;
53      4 : }
54      0 : void TX_PATHType::handle_TXFIFO_insufficient_elements() {
55        :   // FIXME: Default implementation. Implement this.
56      0 : }
57      0 : void TX_PATHType::handle_TXFIFO_overflow() {
58        :   // FIXME: Default implementation. Implement this.
59      0 : }
60      0 : void TX_PATHType::handle_TXFIFO_forced_pop(unsigned int& value) {
61        :   // FIXME: Default implementation. Implement this.
62      0 : }
```

15. After you change the unit test or the model source code, rebuild the model (if necessary) and re-run the unit test to update the report. Refresh the html file and review.

Task-4: Functional Coverage

***Motivation**: In order to validate the quality and completeness of the tests written for a model, TLM Creator provides an automated functional coverage. Since a lot of the model functionality is defined through the use of modeling objects, the traditional line and function coverage does not provide an accurate measure whether all aspects of the specification have been tested.*
For more details on the Functional Coverage, refer to the TLMC Training Lecture Part 2 – "Test Driven Design" and Module 1 Lab B - "Test Driven Design with TLM Creator".

The functional coverage is automatically enabled in TLM Creator. By running the unit tests the functional coverage report is generated and you can view and check which TLM components, i.e., interfaces, memories, registers, bitfields, parameters, etc. are tested and which are not. The basic coverage metrics come 'for free', that is, you do not need to add anything to the code and still will get meaningful coverage results.

16. To view the coverage results, expand `SPICtrl/Tests/Unittests` in ***Project Explorer*** and double-click on the `functional_coverage.html` file.

file:///localdev/poojaa/work/workspace/SPICtrl/Tests/Unittests/functional_coverage.html

Result	Failed		
# Coverpoints to be reviewed	16	Reviewed Coverage:	58.1081%
# Coverpoints to be tested	0	Tested Coverage:	58.1081%
Exempt # Coverpoints	0		
Total # Coverpoints	37		

Unreviewed, uncovered coverpoint	Coverage
spi_clk	0%
clk_in	0%
tfff_irq	0%

Memory Coverpoints

Coverpoint	Bin	Score	Count	Definition	Description
mRegs		disabled			
MCR		disabled			
CLR_RXF		50%	6	WRITES_AND_TRANSACTION_READS	
	default	yes	6	0	
	other	no	0	f0()	
CLR_TXF		50%	6	WRITES_AND_TRANSACTION_READS	
	default	yes	6	0	
	other	no	0	f0()	
MDIS		50%	6	WRITES_AND_TRANSACTION_READS	
	default	yes	6	0	
	other	no	0	f0()	
OFL_RXF		50%	6	WRITES_AND_TRANSACTION_READS	
	default	yes	6	0	
	other	no	0	f0()	
UFL_TXF		50%	6	WRITES_AND_TRANSACTION_READS	
	default	yes	6	0	
	other	no	0	f0()	
PCSSE		100%	6	WRITES_AND_TRANSACTION_READS	

For more details on the Functional Coverage, refer to the TLMC Training Lecture Part 2 – “Test Driven Design” and Module 1 Lab B - “Test Driven Design with TLM Creator”.